

Java multithreading

Threads and why they exist, how we create and schedule them, the lifecycle, and the traps like deadlock, livelock, and race conditions. Definitions stay tight; the experience question is fully spoken.

Contents

1. [What is multithreading, and why do we need it?](#)
2. [Process v/s thread](#)
3. [Multitasking v/s multithreading](#)
4. [Single-threaded v/s multi-threaded applications](#)
5. [Explain the lifecycle of a thread](#)
6. [How does Java support thread creation, and what are the ways?](#)
7. [Difference between start\(\) and run\(\)](#)
8. [How does Java support thread scheduling, and how does it work?](#)
9. [OS scheduler v/s JVM scheduler](#)
10. [Difference between wait\(\) and sleep\(\)](#)
11. [What is a daemon thread, and how does it differ from a user thread?](#)
12. [What is a deadlock, and what conditions are required for one?](#)
13. [What is a livelock, and how does it differ from a deadlock?](#)
14. [What is ThreadLocal, and how does it work internally?](#)
15. [Describe a production issue caused by multithreading](#)

1. What is multithreading, and why do we need it?

Multithreading is a Java program's ability to **run several threads concurrently inside the same process**. Each thread is an independent path of execution, and they all share the process's memory.

```
class Task extends Thread {
    @Override
    public void run() {
        System.out.println(Thread.currentThread().getName());
    }
}

public class Main {
    public static void main(String[] args) {
        Task t1 = new Task();
        Task t2 = new Task();

        t1.start();
        t2.start();
    }
}
```

Possible output:

Thread-0
Thread-1

Why we need it: without multithreading, a program runs one task at a time. Say three jobs run one after another:

- Download a file: 10 sec
- Process the file: 5 sec
- Send an email: 2 sec

That is 17 seconds end to end. With multithreading, independent jobs run at the same time instead:

- Thread-1 downloads a file
- Thread-2 processes another file
- Thread-3 sends an email

So independent tasks overlap rather than queue up, which improves both responsiveness and throughput.

Advantages:

Advantage	Explanation
Improved responsiveness	One slow task does not block the whole application
Better CPU utilisation	Different threads can run on different cores at once
Higher throughput	More work finishes in the same amount of time
Parallel processing	Independent tasks run together on a multi-core system
Efficient resource sharing	Threads share the same memory, so passing data between them is easier than between separate processes

2. Process v/s thread.

A **process** is an independent program in execution, with its own memory space and system resources. A **thread** is the smallest unit of execution inside a process, sharing that process's memory and resources with the other threads.

Feature	Process	Thread
Definition	An independent executing program	A lightweight execution unit inside a process
Memory	Its own memory space	Shares the process memory
Resource sharing	No	Yes
Communication	IPC (inter-process communication)	Shared memory
Creation cost	High	Low
Context switching	Slower	Faster
Failure impact	One process usually does not affect others	One thread can bring down the whole process
Example	Chrome, VS Code, Spotify	Chrome tab renderer, background worker, UI thread

3. Multitasking v/s multithreading.

Multitasking is the operating system running several processes or applications at once. **Multithreading** is a single process running several threads at once.

Feature	Multitasking	Multithreading
Unit of execution	Processes	Threads
Managed by	Operating system	JVM and operating system
Memory	Each process has its own memory	Threads share the process memory
Resource sharing	No	Yes
Context switching	More expensive	Less expensive
Communication	IPC (pipes, sockets, and so on)	Shared memory
Example	Chrome, Spotify, and VS Code together	Several threads inside one Spring Boot application

In short: multitasking is many applications running together, multithreading is many threads running inside one application.

4. Single-threaded v/s multi-threaded applications.

A **single-threaded** application runs one task at a time on a single thread. A **multi-threaded** application uses several threads to run tasks concurrently within the same process.

Feature	Single-threaded	Multi-threaded
Threads	One	Several
Task execution	One task at a time	Many tasks concurrently
Responsiveness	One slow task blocks the app	Other threads keep working
CPU utilisation	Limited	Uses multi-core CPUs better
Complexity	Simple	Harder (synchronisation, thread safety)
Performance	Fine for simple tasks	Better for concurrent or I/O-heavy work

Single-threaded: each task starts only after the previous one finishes.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Task 1");
        System.out.println("Task 2");
        System.out.println("Task 3");
    }
}
```

Multi-threaded: the tasks run concurrently.

```
class Task extends Thread {
    private final String taskName;

    public Task(String taskName) {
        this.taskName = taskName;
    }

    @Override
    public void run() {
        System.out.println(taskName);
    }
}

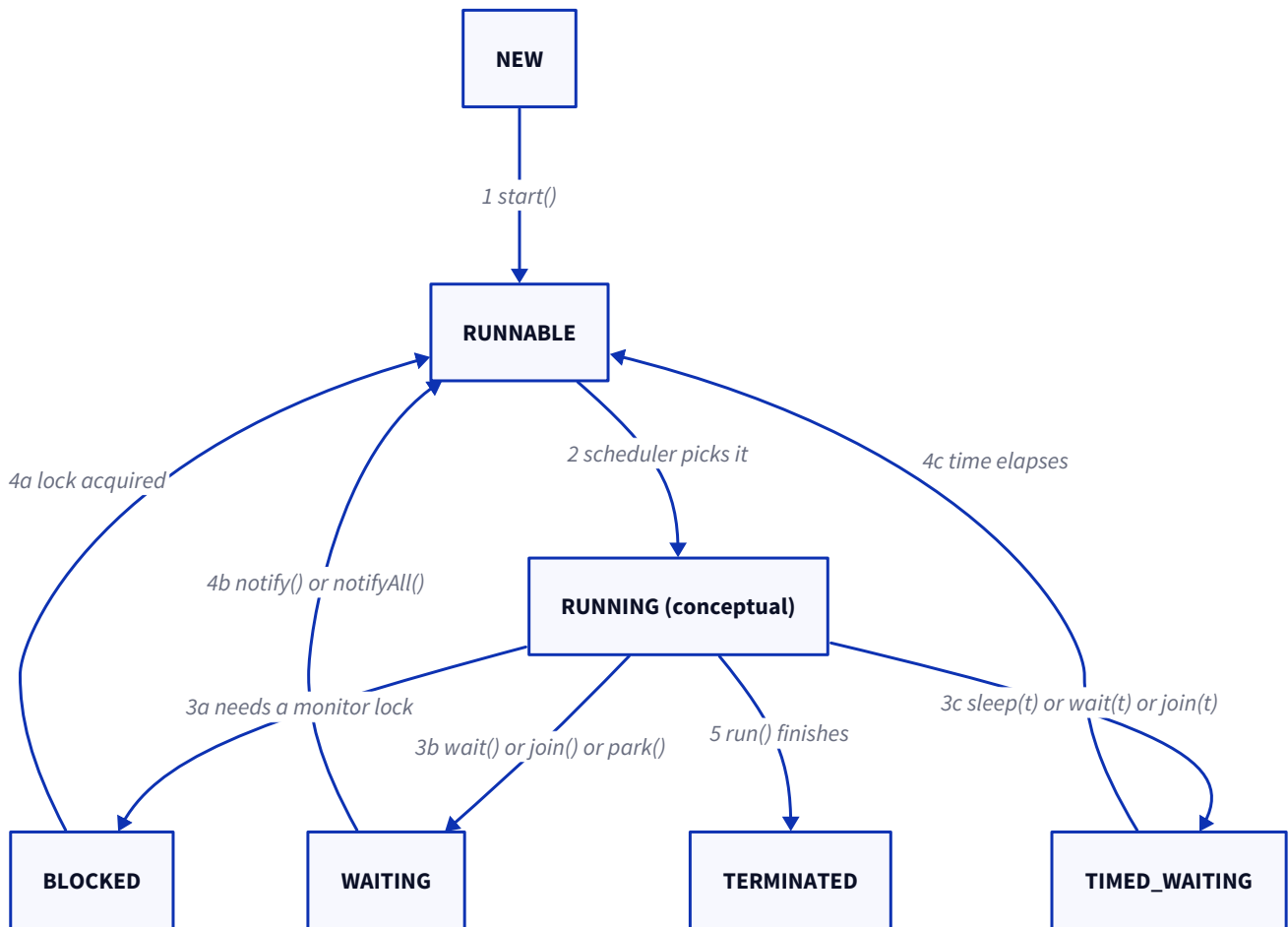
public class Main {
    public static void main(String[] args) {
        new Task("Task 1").start();
        new Task("Task 2").start();
        new Task("Task 3").start();
    }
}
```

Where each wins:

- Single-threaded: simple to build and debug, no synchronisation issues, no race conditions or deadlocks.
- Multi-threaded: better responsiveness, better CPU use, higher throughput, and it can handle many requests at once.

5. Explain the lifecycle of a thread.

The lifecycle is the set of states a thread moves through from creation to termination. In Java these are defined by the `Thread.State` enum.



WORTH CLARIFYING

The `Thread.State` enum has no separate **RUNNING** state. It uses **RUNNABLE** for a thread that is either ready to run or actually running on the CPU. Interviewers often talk about "running" as a concept, but it is not an official enum value.

The states, one by one:

NEW: the thread object is created but not started yet.

```
Thread thread = new Thread();
```

RUNNABLE: after `start()`, the thread is ready to execute and waiting for the CPU scheduler to pick it.

```
thread.start();
```

RUNNING (conceptual): once the scheduler selects it, its `run()` method executes.

BLOCKED: the thread is waiting to acquire a monitor lock held by another thread.

```
synchronized (lock) {  
    // another thread already owns the lock  
}
```

WAITING: the thread waits indefinitely until another thread wakes it.

```
object.wait();    // or thread.join(); or LockSupport.park();
```

TIMED_WAITING: the thread waits, but only for a set time.

```
Thread.sleep(5000);    // or thread.join(1000); or object.wait(1000);
```

TERMINATED: the thread has finished `run()`, or exited because of an uncaught exception.

6. How does Java support thread creation, and what are the ways?

Java gives us the `Thread` class plus the `Runnable` and `Callable` interfaces. There are a few APIs, but the approach I would reach for today is to write a `Runnable` or `Callable` and hand it to an `ExecutorService`, rather than creating threads by hand.

Approach	Recommended	Use case
Extend <code>Thread</code>	No	Simple examples and learning
Implement <code>Runnable</code>	Yes	The most common approach
Implement <code>Callable</code>	Yes	Tasks that return a result or throw checked exceptions
<code>ExecutorService</code> (thread pool)	Best	Production applications

1. Extend the `Thread` class: override `run()` and call `start()`.

```
class MyThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println("Running...");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MyThread thread = new MyThread();  
        thread.start();  
    }  
}
```

Drawbacks: Java has no multiple inheritance, so extending `Thread` uses up our one superclass, and the task ends up tightly coupled to the thread.

2. Implement `Runnable` (recommended): implement the interface and pass it to a `Thread`.

```

class Task implements Runnable {
    @Override
    public void run() {
        System.out.println("Running...");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread thread = new Thread(new Task());
        thread.start();
    }
}

```

Advantages: the task and the thread stay separate, the class can still extend something else, and it is what most Java code uses.

3. Implement Callable : use it when the task needs to return a value or throw a checked exception.

```

Callable<Integer> task = () -> 10;

ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> future = executor.submit(task);

System.out.println(future.get());
executor.shutdown();

```

Advantages: it returns a result, it can throw checked exceptions, and it works with Future .

4. Use ExecutorService (production standard): submit tasks to a thread pool instead of creating threads ourselves.

```

ExecutorService executor = Executors.newFixedThreadPool(5);

executor.submit(() -> {
    System.out.println("Task running...");
});

executor.shutdown();

```

Advantages: it reuses threads, performs better, caps how many threads get created, and takes the thread management off our hands.

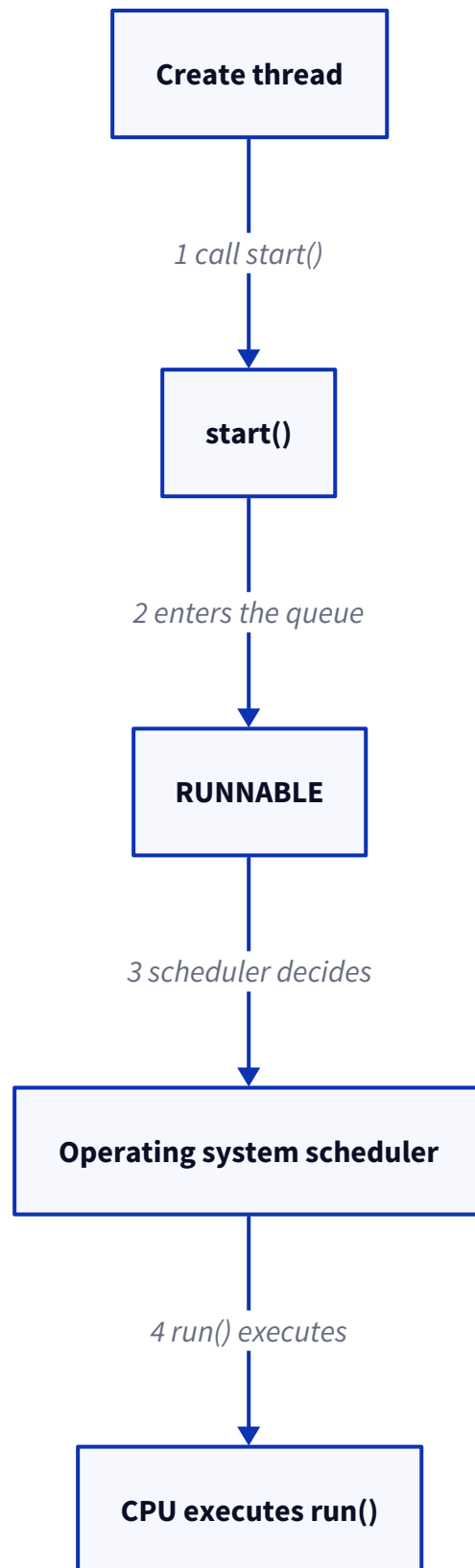
7. Difference between start() and run().

start() creates a new thread and asks the JVM to run run() on that thread. run() on its own is just a normal method: calling it directly runs the code on the current thread and creates no new thread at all.

Feature	start()	run()
Creates a new thread	Yes	No
Executed by	JVM / thread scheduler	The current thread
Calls <code>run()</code> internally	Yes	No
Runs concurrently	Yes	No
Can be called many times	No	Yes (like any normal method)

8. How does Java support thread scheduling, and how does it work?

Thread scheduling is how the JVM and the operating system decide which runnable thread gets CPU time and when. Java does not ship its own scheduler; it leans on the operating system's thread scheduler underneath.



So when we call `thread.start()`, the thread does not run straight away. It enters the `RUNNABLE` state, and the OS scheduler decides when it actually gets the CPU.

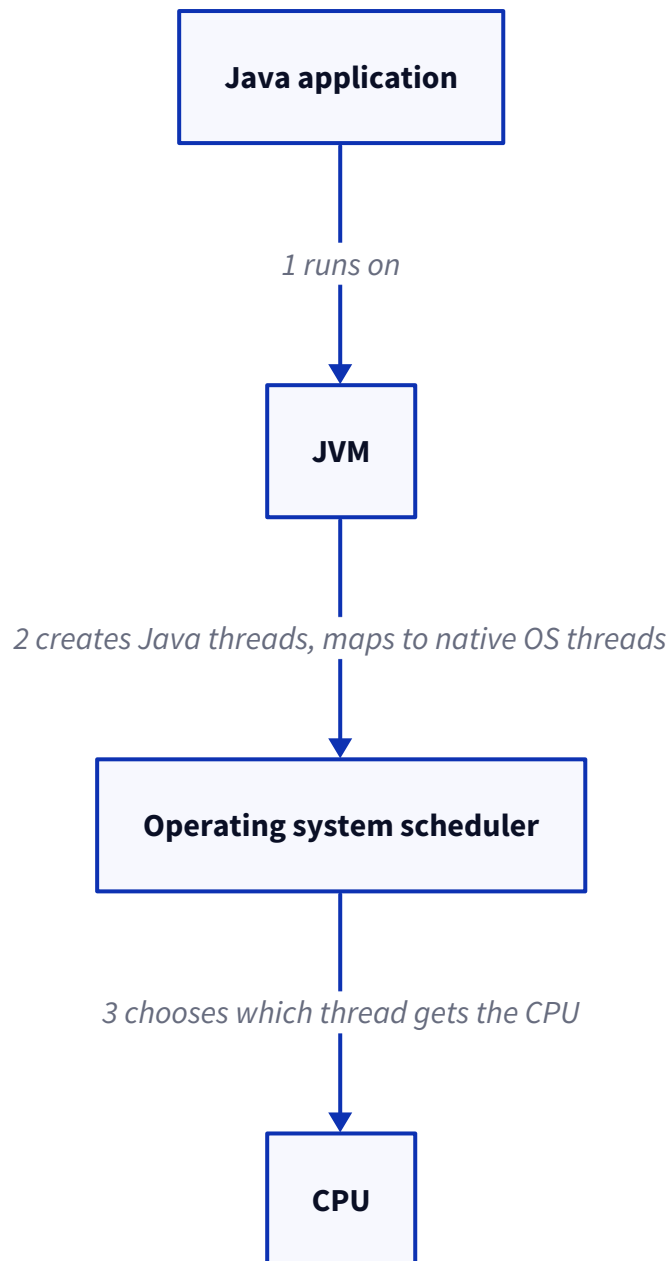
What Java gives us to influence scheduling: these nudge the scheduler, but none of them guarantee an execution order.

Method	Purpose
<code>Thread.start()</code>	Makes the thread eligible for scheduling
<code>Thread.sleep()</code>	Pauses the current thread for a set time
<code>Thread.yield()</code>	Hints that the current thread will let others run
<code>Thread.join()</code>	Waits for another thread to finish
<code>setPriority()</code>	Suggests a scheduling priority to the OS

9. OS scheduler v/s JVM scheduler.

Modern Java has no scheduler of its own. The JVM creates and manages the Java threads, but the operating system's scheduler decides when those threads actually run on the CPU.

Feature	JVM	OS scheduler
Creates Java threads	Yes	No
Manages the thread lifecycle	Yes	No
Maps Java threads to native OS threads	Yes	No
Decides which thread gets CPU time	No	Yes
Performs context switching	No	Yes
Uses thread priorities	Passes the priority to the OS	May consider it, not guaranteed



10. Difference between wait() and sleep().

Both pause a thread, but for different reasons. `sleep()` pauses a thread for a set time and **holds on to any locks**. `wait()` pauses a thread and **releases the monitor lock** until another thread notifies it.

Feature	<code>wait()</code>	<code>sleep()</code>
Belongs to	<code>Object</code>	<code>Thread</code>
Purpose	Thread communication	Pause execution for a fixed time
Releases the lock	Yes	No
Must be inside a <code>synchronized</code> block	Yes	No
Wakes up on	<code>notify()</code> , <code>notifyAll()</code> , <code>interrupt</code> , or <code>timeout</code>	Timeout expiring or an <code>interrupt</code>
Time parameter	Optional (<code>wait()</code> or <code>wait(long)</code>)	Mandatory (<code>sleep(long)</code>)
Throws	<code>InterruptedException</code>	<code>InterruptedException</code>

11. What is a daemon thread, and how does it differ from a user thread?

A daemon thread is a **background thread that supports the user threads**. The JVM shuts daemon threads down automatically once all the user (non-daemon) threads have finished.

Feature	User thread	Daemon thread
Purpose	Does the application's main work	Does background or support tasks
Keeps the JVM alive	Yes	No
JVM exit	The JVM waits for user threads	The JVM does not wait for daemon threads
Examples	The <code>main()</code> thread, request-processing threads	Garbage collector, monitoring, cleanup tasks

```

class MyDaemon extends Thread {
    @Override
    public void run() {
        while (true) {
            System.out.println("Daemon thread running...");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                break;
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyDaemon daemon = new MyDaemon();
        daemon.setDaemon(true); // must be called before start()
        daemon.start();

        System.out.println("Main thread finished");
    }
}

```

Possible output:

```

Main thread finished
Daemon thread running...

```

The JVM may exit right after the main thread finishes, even if the daemon thread is still mid-loop. That is the whole point of a daemon: it never keeps the process alive on its own.

12. What is a deadlock, and what conditions are required for one?

A deadlock is when two or more threads are **blocked forever**, each waiting for a resource the other is holding. Nobody can move, so nobody ever finishes.

```

class Main {
    private static final Object LOCK1 = new Object();
    private static final Object LOCK2 = new Object();

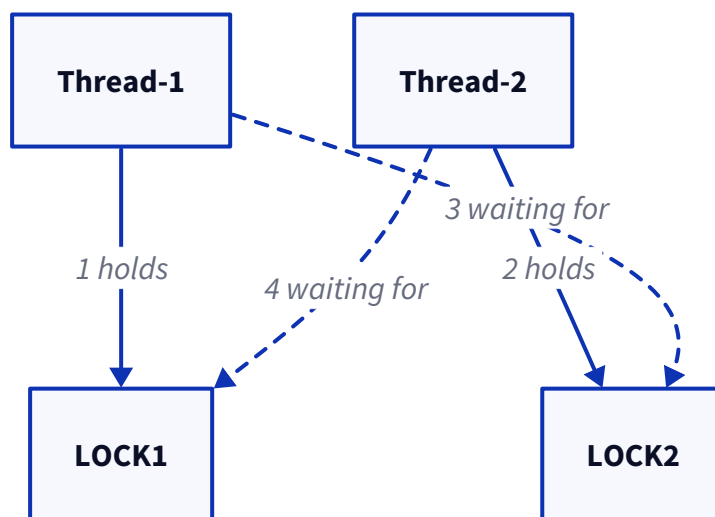
    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (LOCK1) {
                System.out.println("Thread 1 acquired LOCK1");
                synchronized (LOCK2) {
                    System.out.println("Thread 1 acquired LOCK2");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (LOCK2) {
                System.out.println("Thread 2 acquired LOCK2");
                synchronized (LOCK1) {
                    System.out.println("Thread 2 acquired LOCK1");
                }
            }
        });

        t1.start();
        t2.start();
    }
}

```

Thread 1 holds LOCK1 and waits for LOCK2. Thread 2 holds LOCK2 and waits for LOCK1. Each is waiting on a lock the other will never release, so both wait forever.



Conditions required: all four of these must hold at the same time for a deadlock to happen. Break any one and the deadlock cannot form.

- **Mutual exclusion:** a resource can be held by only one thread at a time.
- **Hold and wait:** a thread holds one resource while waiting to acquire another.
- **No preemption:** a resource cannot be forcibly taken; the holder must release it voluntarily.
- **Circular wait:** a cycle of threads exists where each waits for a resource the next one holds.

13. What is a livelock, and how does it differ from a deadlock?

A livelock is when two or more threads stay active and keep reacting to each other, but none of them makes any real progress. Unlike a deadlock, the threads are not blocked: they keep running, they just accomplish nothing.

Feature	Deadlock	Livelock
Thread state	Blocked	Running and active
Makes progress	No	No
CPU usage	Low	High
Waiting on	A lock or resource	Constantly retrying or reacting
Recovery	Usually needs outside intervention	Often fixed by changing the retry or backoff strategy
Example	Two threads waiting for each other's locks	Two threads that keep releasing and reacquiring resources

14. What is ThreadLocal, and how does it work internally?

ThreadLocal gives us a **per-thread variable**: each thread gets its own independent copy, and a change one thread makes is invisible to the others.

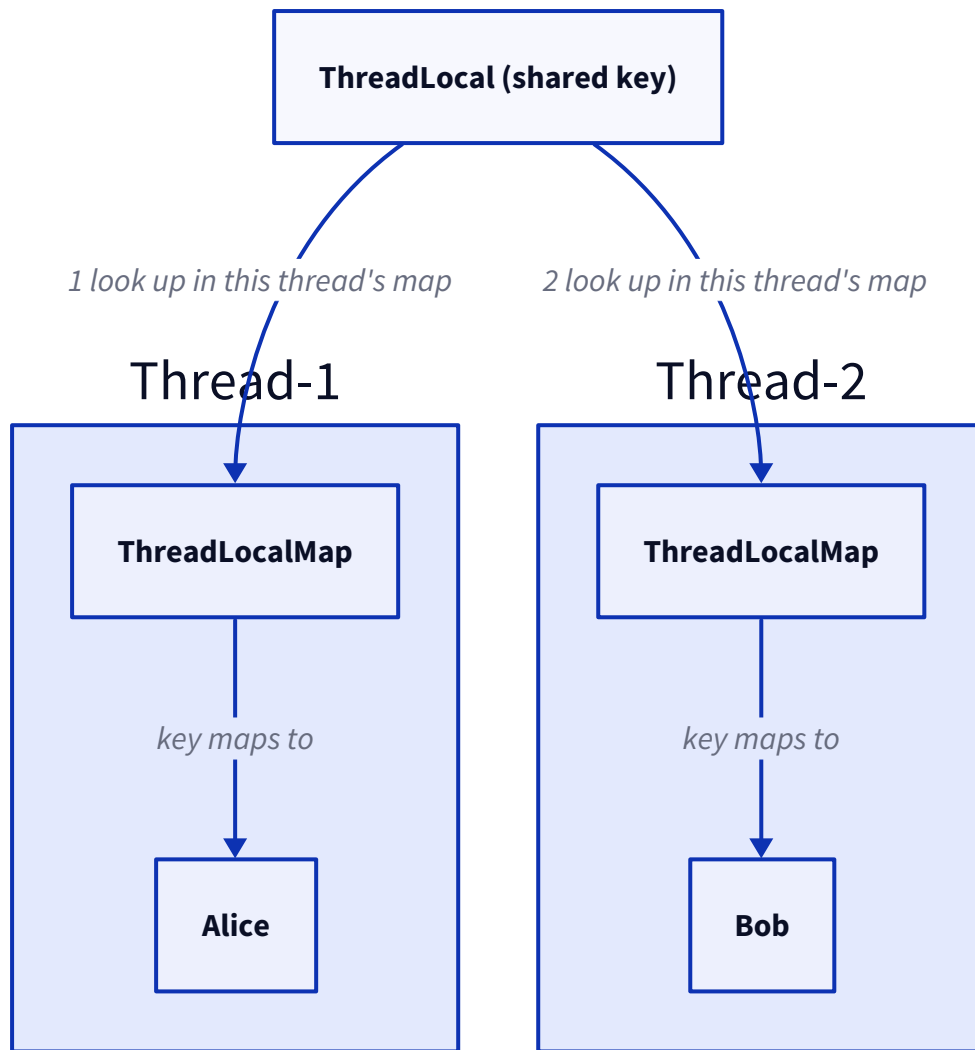
Why we need it: if several threads share the same object, they all touch the same field, which invites race conditions.

```
class UserContext {
    String userId; // shared by every thread
}
```

With ThreadLocal, each thread gets its own copy instead.

```
ThreadLocal<String> userId = new ThreadLocal<>();
```

How it works internally: this is the part people get backwards. The common mental model is that the ThreadLocal holds a map of thread to value. It is actually the opposite. Each Thread owns a map called ThreadLocalMap, and the ThreadLocal is just the key used to look up a value in that map.



So the `ThreadLocal` does not store the value. The `Thread` does. That is also why a value set on one thread never leaks to another, and why we should call `remove()` on thread-pool threads, since the map outlives any single task.

15. Describe a production issue caused by multithreading. How did you resolve it?

So we had a Spring Boot service that would now and then create duplicate records, but only when traffic was high. The frustrating part was that it never showed up in development, because it only surfaced when several requests hit the same path at the same instant.

The way I went at it, I started with the application logs and the request IDs, then lined those up against the database records and their timestamps. What that showed was two threads running the same check-then-insert logic at once. Both checked, both saw the record was missing, and both went ahead and inserted it.

The code looked like this:

```
if (!repository.exists(id)) {
    repository.save(entity);
}
```

That is the whole problem: it is not atomic. The check and the insert are two separate steps, so two threads can slip in between them. A classic race condition.

For the fix I did not lean on a single safeguard. We added a unique constraint in the database as the real backstop, wrapped the operation in a transaction where it made sense, and replaced the check-then-act with an atomic write, letting the constraint reject the duplicate and handling the duplicate-key exception. Then I added concurrent load tests, so a regression would surface in CI rather than in production.

After that the duplicates stopped, the service held up under load, and we put monitoring in place to catch anything similar early. The lesson I took from it: under concurrency, a check-then-act is only as safe as the atomic guarantee underneath it, so push the safety down to the database or an atomic operation rather than trusting application-level ordering.